AI Use Case Implementation Using JamAI Base

Prepared by: Tan Chee Keat | 25006123 | Universiti Malaya |WIA1006 OCC7

1. Rationale and Use Case Identification

The chosen use case is automating e-invoicing compliance for Micro, Small, and Medium Enterprises (MSMEs) in Malaysia. This targets the Finance and Business Services sector to help businesses adapt to the new Malaysian Inland Revenue Board (LHDN) mandate.

1.1 The LHDN Mandate and Challenges

Starting July 2026, businesses with annual revenues between RM1 million and RM5 million must issue e-invoices. While businesses earning under RM1 million are generally exempt, many are still forced to comply due to specific conditions, such as having corporate shareholders or operating as a subsidiary. Additionally, any single transaction exceeding RM10,000 cannot be grouped and requires a separate e-invoice.

Although the government provides a free MyInvois Portal, the manual process is slow. Business owners spend about 15 minutes per invoice typing in details like Tax Identification Numbers (TIN) and MSIC codes. This manual data entry is highly prone to human error, which can lead to compliance penalties.

1.2 The EzBill Solution

EzBill is an AI tool that solves this problem. Users simply upload a photo of a physical receipt. The system uses AI to read the text, apply the correct tax rules, and format the data into the required UBL 2.1 JSON file. This reduces processing time to under 30 seconds and minimizes data entry errors.

2. Backend Implementation Using JamAI Base

The backend is built entirely on JamAI Base using an Action Table. This setup processes unstructured images into structured data in a single pipeline. To ensure accuracy, the AI uses Retrieval-Augmented Generation (RAG) to check official rules.

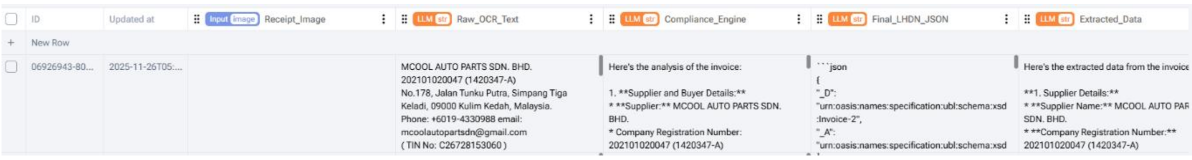
2.1 Knowledge Tables (RAG)

- **LHDN_Knowledge_Base & LHDN_Knowledge:** Stores official government guidelines, tax codes, and rules (like the RM10,000 limit).

- **Sample_JSON:** Stores valid UBL 2.1 templates to ensure the output format is exactly what the LHDN API expects.

2.2 Action Table Configuration

Column Name	Data Type	Function & Logic
Receipt_Image	Input (Image)	Accepts the receipt image file from the user.
Raw_OCR_Text	Output (LLM)	Reads and lists every line of text from the image exactly as it appears. Separating this step prevents the AI from inventing false information.
Compliance_Engine	Output (LLM) **RAG Enabled**	Analyzes the extracted text. It references the *LHDN_Knowledge* tables to identify buyer details, find the correct MSIC code, and determine tax rules. It outputs logic, not the final file format.
Final_LHDN_JSON	Output (LLM) **RAG Enabled**	Uses the logic from the previous step and the *Sample_JSON* table to build the strict UBL 2.1 JSON format required by LHDN.
Extracted_Data	Output (LLM)	Creates a simple text summary of the invoice data to display on the user interface.



3. Reflection and Outcomes

3.1 Outcomes Achieved

- **Time Saved:** The automated pipeline cuts invoice processing time from 15 minutes to under 30 seconds.
- **Improved Accuracy:** Extracting exact text before applying logic prevents the AI from guessing missing details.
- **Rule Compliance:** Using RAG with official LHDN documents ensures the AI correctly classifies complex tax scenarios.

3.2 Limitations and Future Improvements

- **Image Quality Issues:** Faded, crumpled, or blurry receipts reduce the accuracy of the text extraction step.
- **Processing Delay:** The step-by-step AI pipeline takes 30 to 40 seconds to finish. While faster than typing, this delay is not ideal for busy retail checkout counters.
- **Context Limits:** Very rare tax situations sometimes require extra prompt tuning to make sure the AI retrieves the right rule from the Knowledge Table.